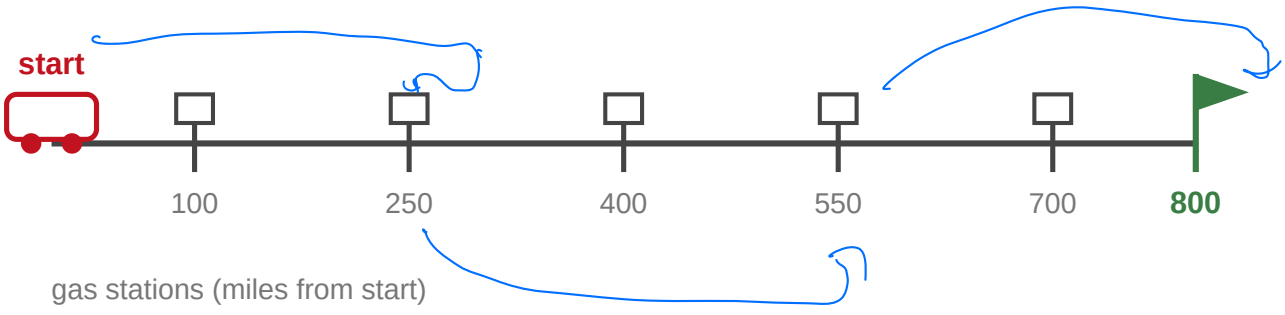
Greedy algorithms

Trip planning

A full tank takes you 300 miles. Every station fills you back to full.



Tweak: the stations aren't all equal

Each station now gives a **different** amount of fuel.

previous strat: As far as you can → no longer optimal

A counterexample

You start with 30 miles of fuel. Stations now give different amounts.

Go as far as you can -- stop at mile 30



can now reach mile 31

Stop early -- stop at mile 10



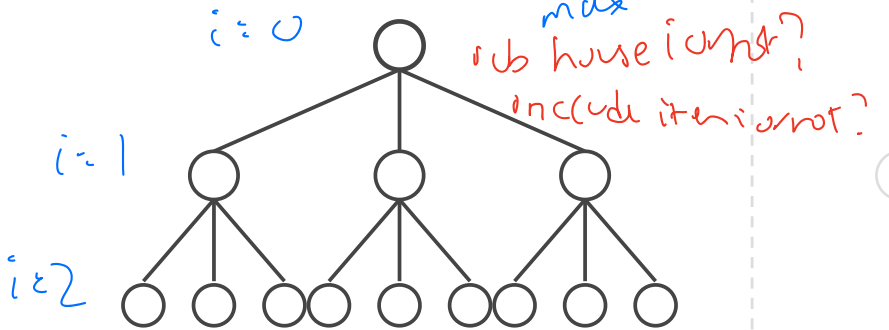
can now reach mile 1030

Greedy sometimes works

Greedy vs DP

DP

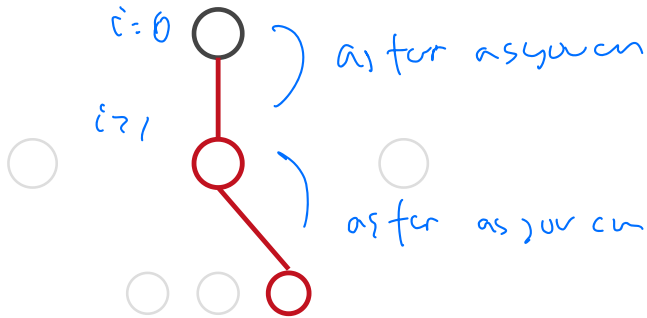
try every option, remember the answers



explores everything -- but caches

Greedy

one rule picks the move, then never look back



commits to one path -- no going back

The greedy cookbook

1. **Brainstorm** greedy strategies / how to find the next best move.
2. **Prove**, or kill it with a **counterexample**
3. **Code** it

hardest part!

Trip planning, formalized

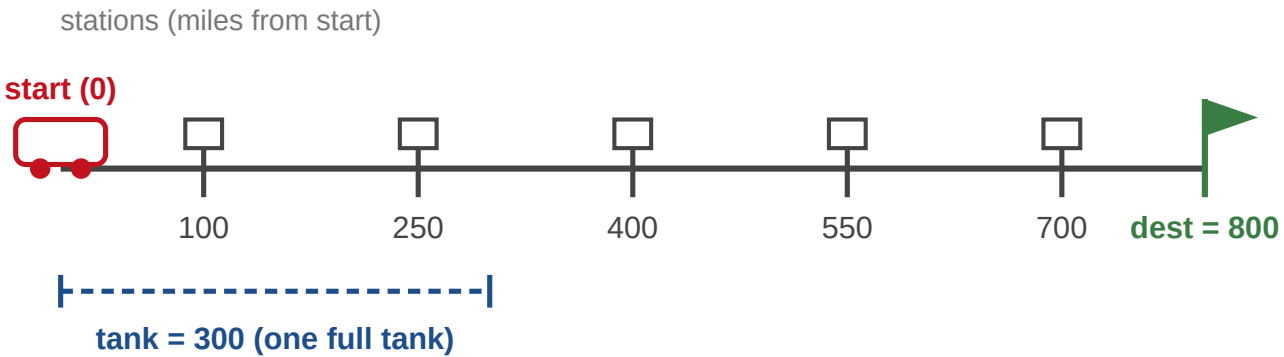
Input

- `stations`. an array of distances from the start
- `tank`. how many miles one full tank covers
- `dest`. the distance to the destination

Output: the minimum number of refuelling stops, or -1 if you cannot get there.

Trip planning, formalized

- `stations = [100, 250, 400, 550, 700]` (mi)
- `tank = 300`
- `dest = 800`



Q: when to return -1?

A: when... any stations[i+1] - stations[i] > tank

Output: ?

Step 1. Brainstorm greedy strategy

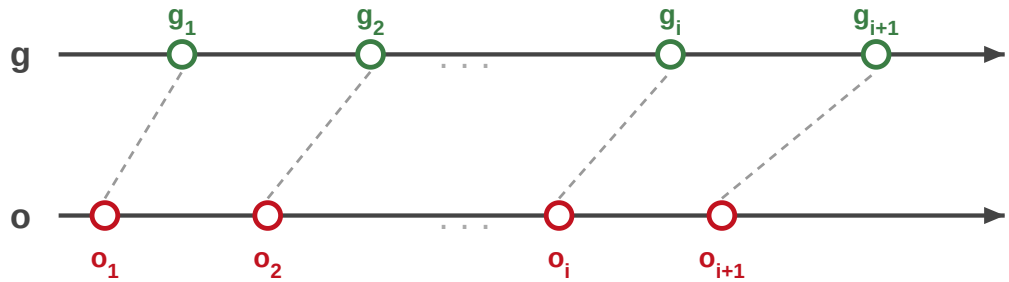
- **Nearest first.** Stop at the first station you meet.
- **Furthest first.** Drive as far as the tank allows, then stop.

goal. greedy strat is optimal

hard.

Step 2. Prove: greedy stays ahead

at every step i, greedy is always ≥ any other strat.



g_i = distance travelled after i steps if we follow greedy approach

o_i = — " — optimal approach

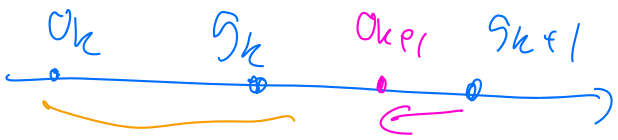
Induction proof.

Goal: $g_i \geq o_i \quad \forall i$

Base case: $i = 1$, yes. because. g_1 = furthest you can.

I.S. assuming $g_k \geq o_k$, show: $g_{k+1} \geq o_{k+1}$

on kth stop,



math it up

$o_{k+1} \leq o_k + \text{TANK} \overset{?}{\leq} g_k + \text{tank}$

because.. $o_k \leq g_k$

↓ reachable by g_{k+1}

Written proof

Let g_i be the position of greedy's i-th stop, and o_i the i-th stop of any other valid plan.

Claim. $g_i \geq o_i$ for every i.

Base case. Both plans start with the same full tank, so both can reach exactly the same set of stations. Greedy takes the farthest, so $g_1 \geq o_1$.

Inductive step. Assume $o_i \leq g_i$. The other plan's next stop is at most one tank further, so $o_{i+1} \leq o_i + \mathrm{TANK}$. Chaining the hypothesis onto that, $o_{i+1} \leq o_i + \mathrm{TANK} \leq g_i + \mathrm{TANK}$, so $o_{i+1} \leq g_i + \mathrm{TANK}$. But greedy can reach anywhere between g_i and $g_i + \mathrm{TANK}$, so anywhere o_{i+1} can reach, g_{i+1} can reach as well. Greedy takes the farthest of those, so $g_{i+1} \geq o_{i+1}$.

Conclusion. Greedy is never behind after the same number of stops, so it never needs more stops than the other plan.

Step 3. Code

```
stations.sort()
reach = tank # furthest mile we can get to right now
stops = 0
i = 0
while reach < dest:
    best = -1
    while i < len(stations) and stations[i] <= reach:
        best = stations[i]
        i += 1
    if best < 0:
        return -1 # stranded
    reach = best + tank
    stops += 1
return stops
```

O(n log n)

O(n)

Runtime

Does our proof work if TANK(i) is variable? No

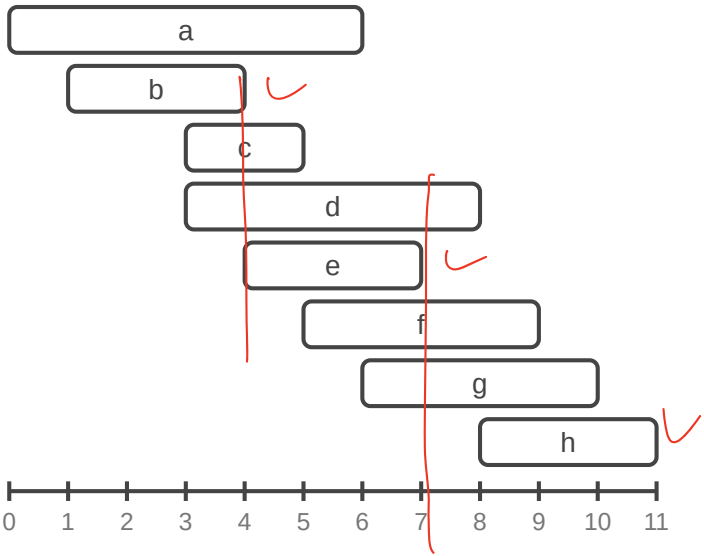
Why proof is broken with variable fuel

Inductive step. Assume $o_i \leq g_i$. The other plan's next stop is at most one tank further, so $o_{i+1} \leq o_i + \text{TANK}$. Chaining the hypothesis onto that, $o_{i+1} \leq o_i + \text{TANK} \leq g_i + \text{TANK}$, so $o_{i+1} \leq g_i + \text{TANK}$. But greedy can reach anywhere between g_i and $g_i + \text{TANK}$, so anywhere o_{i+1} can reach, g_{i+1} can reach as well. Greedy takes the farthest of those, so $g_{i+1} \geq o_{i+1}$.

no longer true

$o_i + \text{TANK}(o_i)$ vs $g_i + \text{TANK}(g_i)$

Interval scheduling



shortest interval first?

3!

Formalized

Input: `intervals`, a list of `(start, end)` pairs.

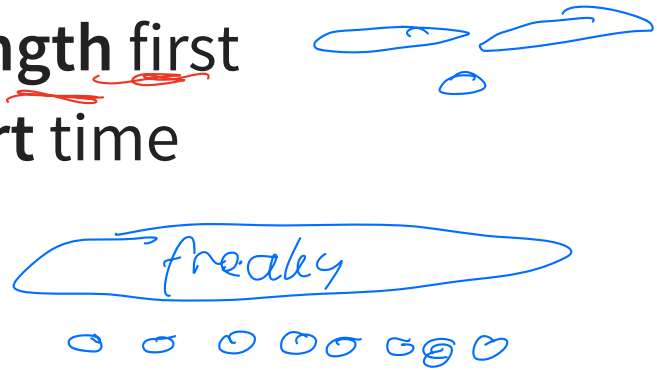
Output: the max number of intervals that pairwise do not overlap.

Step 1. Brainstorm greedy strategy

- Shortest **length** first
- Earliest **start** time
- Earliest **end** time
- None of the above

Step 2. Counter examples

- Shortest **length** first
- Earliest **start** time



Step 2. Proof: Exchange argument

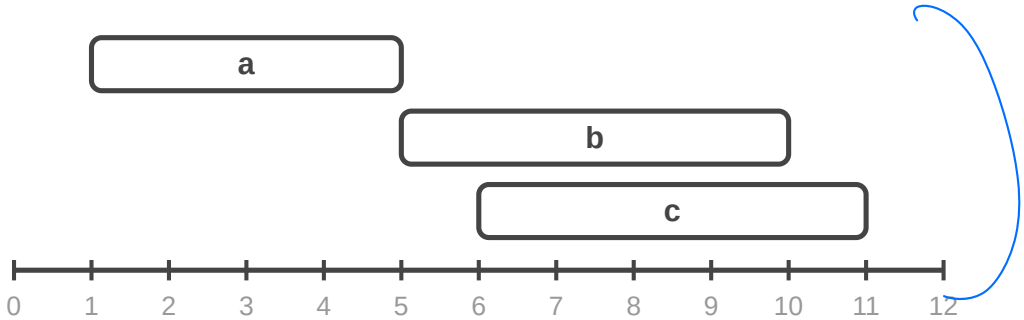
Put **greedy** and **an optimal solution** side by side, both sorted by your strategy.

Walk along them until they **first disagree**.

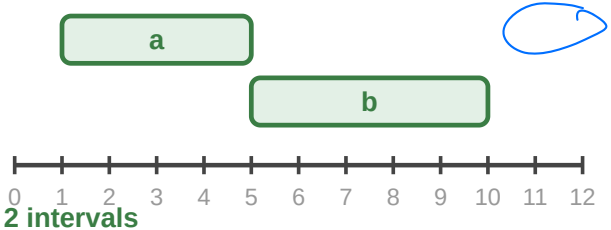
Show the disagreement is **not worse off**.

Repeat until all disagreements are explained.

Exchange argument intuition

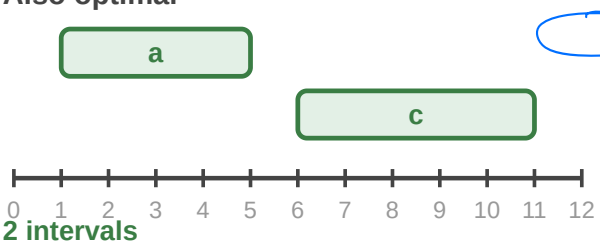


Greedy picks



2 intervals

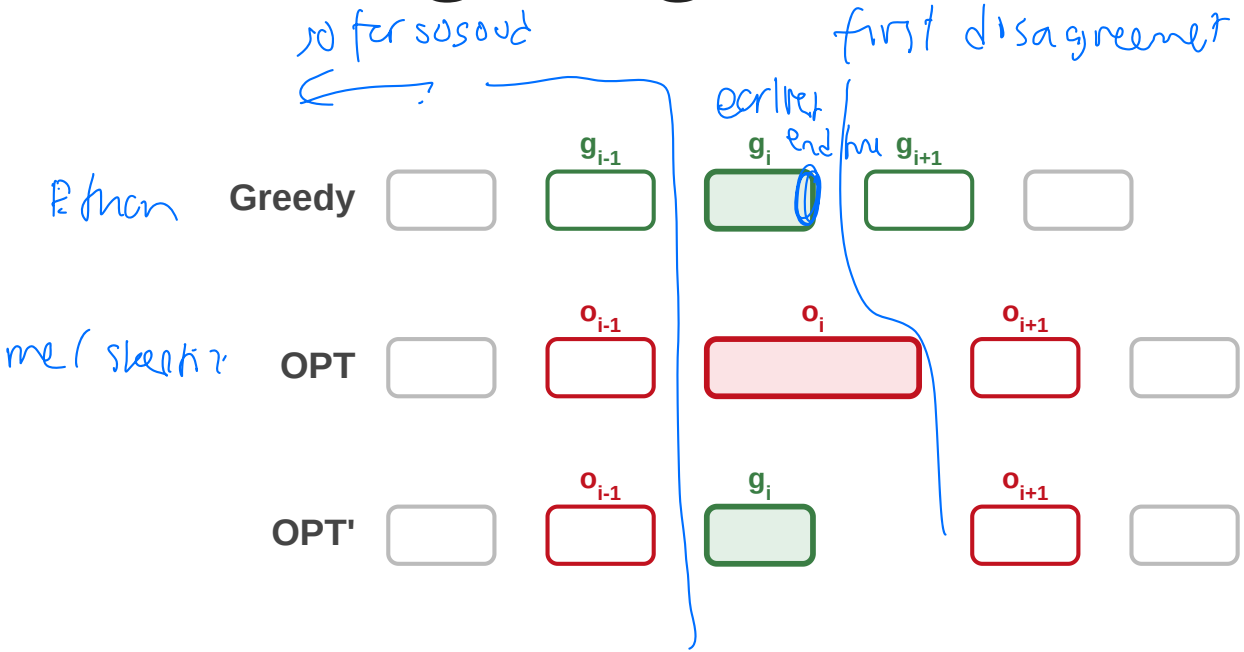
Also optimal



2 intervals

"future concerts that b/c/d/e will still be compatible w/ my "b" choice"

Exchange argument - line up

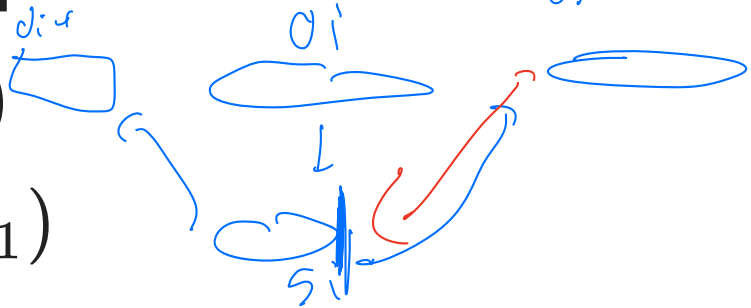


Exchange argument - proof that every swap is valid

this is the first disagreement.

Left. $\text{start}(g_i) \geq \text{end}(o_{i-1})$

Right. $\text{end}(g_i) \leq \text{start}(o_{i+1})$



can never hurt

because: $\text{end}(g_i) \leq \text{end}(o_i) \leq \text{start}(o_{i+1})$

The argument, written out

Let $g_1, \ldots, g_k$ be greedy's intervals and $o_1, \ldots, o_m$ an optimal solution's, both in end-time order.

Claim. If the two agree on the first $i-1$ picks and differ at i, we can replace o_i with g_i and still have a valid solution of the same size.

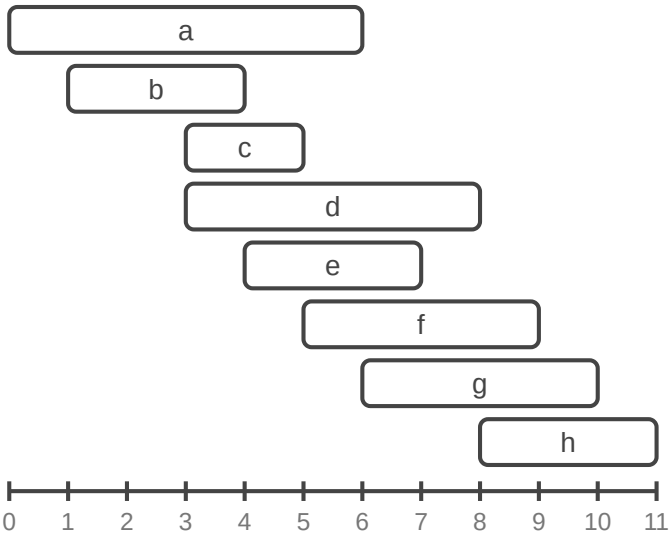
Why the swap is legal. Greedy takes the compatible interval with the smallest end time, so $\mathrm{end}(g_i) \le \mathrm{end}(o_i)$. It does not collide on the left, and it does not collide on the right:

$$\mathrm{start}(g_i) \ge \mathrm{end}(g_{i-1}) = \mathrm{end}(o_{i-1})$$

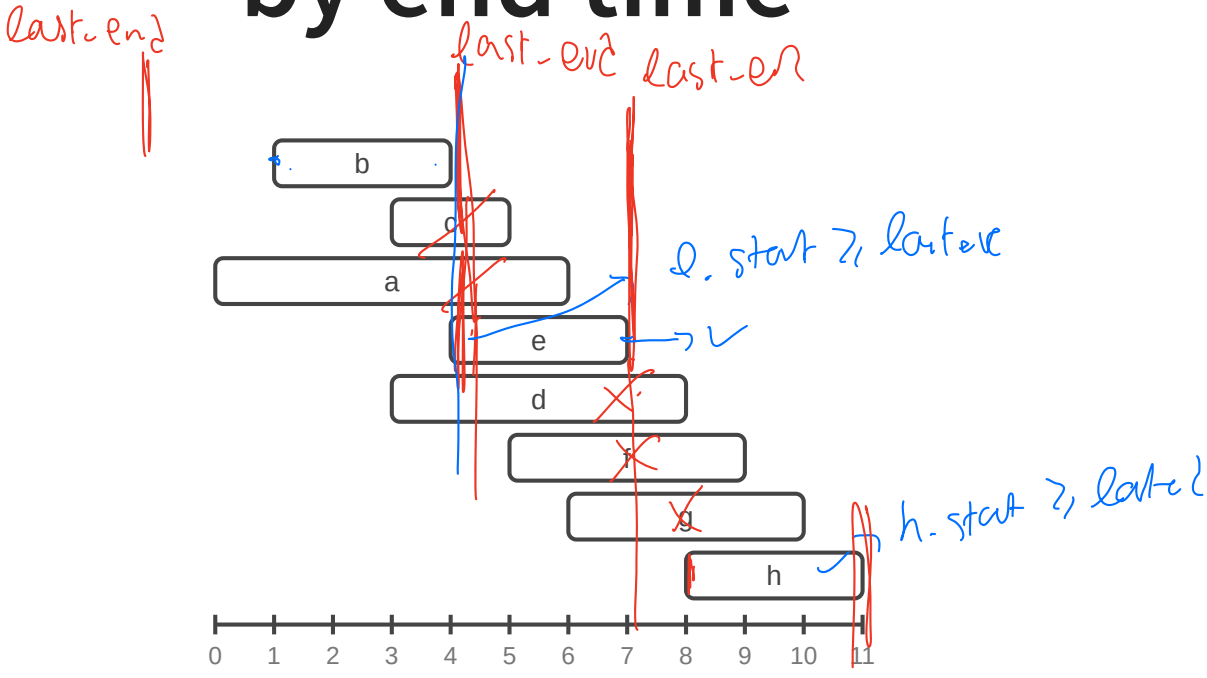
$$\mathrm{end}(g_i) \le \mathrm{end}(o_i) \le \mathrm{start}(o_{i+1})$$

Conclusion. Each swap leaves the count unchanged and extends the agreement by one, so repeating drives the optimal solution to greedy's without ever losing an interval. Therefore $k = m$, and greedy is optimal.

Simulating the solution



Simulating the solution: sorted by end time



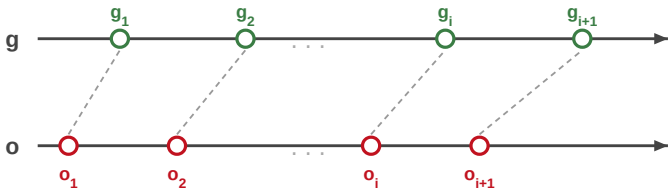
Step 3. Code

```
def max_non_overlapping(intervals):
  intervals.sort(key=lambda iv: iv[1])   # by end time
  count = 0
  last_end = float('-inf')

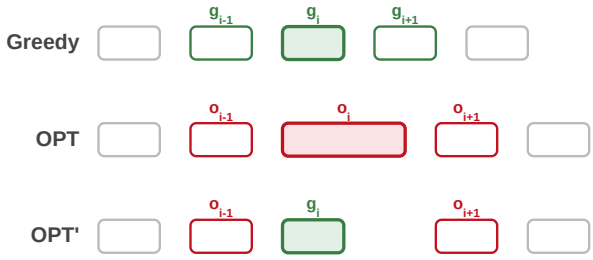
  for start, end in intervals:
    if start >= last_end:
      count += 1
      last_end = end

  return count
```

Two proof styles



Greedy stays ahead -- compare a quantity at every step



Exchange -- bend OPT into greedy, one swap at a time

DP

- brute force subproblems
- every available path
- caching "ecel segment once"
- references subproblem results.

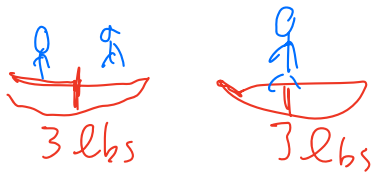
is this better?

Greedy

- no backtracking
- next best available ✓
- not all options

Intuition is k

this is better, let's go



Boats to save people

Everyone must be rescued. Each boat holds **at most two** people and has a weight limit.

```
people = [3, 2, 2, 1], limit = 3
```



if 2 canoes, 1 person is R.I.P.

min # canoes = 3

Formalized

Input: `people`, an array of weights, and `limit`, the weight capacity of one boat.

Output: the minimum number of boats, given each boat carries at most 2 people and at most `limit` total weight.

Strategies

pair randomly / doc about order

- skip too-heavy people
- go through each person
  - → p_i unassigned
  - for each possible partner p_j
    - → find first available

problem clarifications

-  >  exists



w = 1000

[999, 1, 999, 1]

counterexample

1, 1, 999, 999

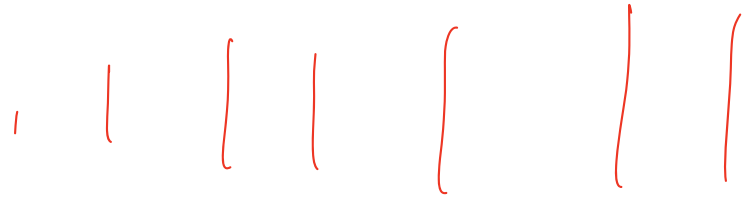


Strategy 2

- sort array: hi → lo
- fit biggest person first
  - → to find partner → find the heaviest person that fits

Surprise:

even the lightest partner, is also optimal



Step 1. Brainstorm greedy strategy

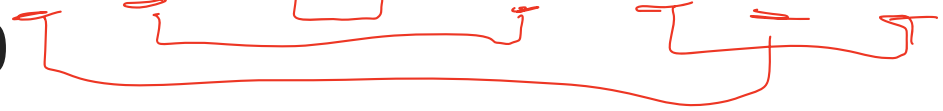
- Pair the two **lightest** people



Step 1. Brainstorm greedy strategy

- Pair the **heaviest** with the heaviest that fits.

```
people = [8, 7, 6, 4, 3, 3, 2, 1]
limit = 10
```

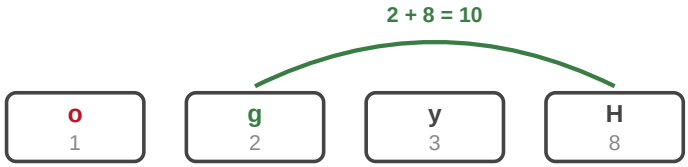


4 canoes

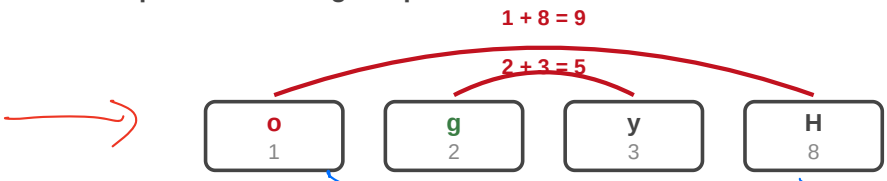


Step 2. Prove - exchange argument

Greedy

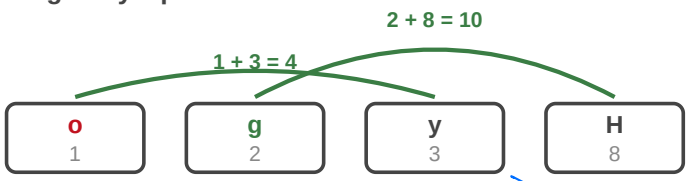


OPT pairs H with a lighter partner



pair H w/ g!

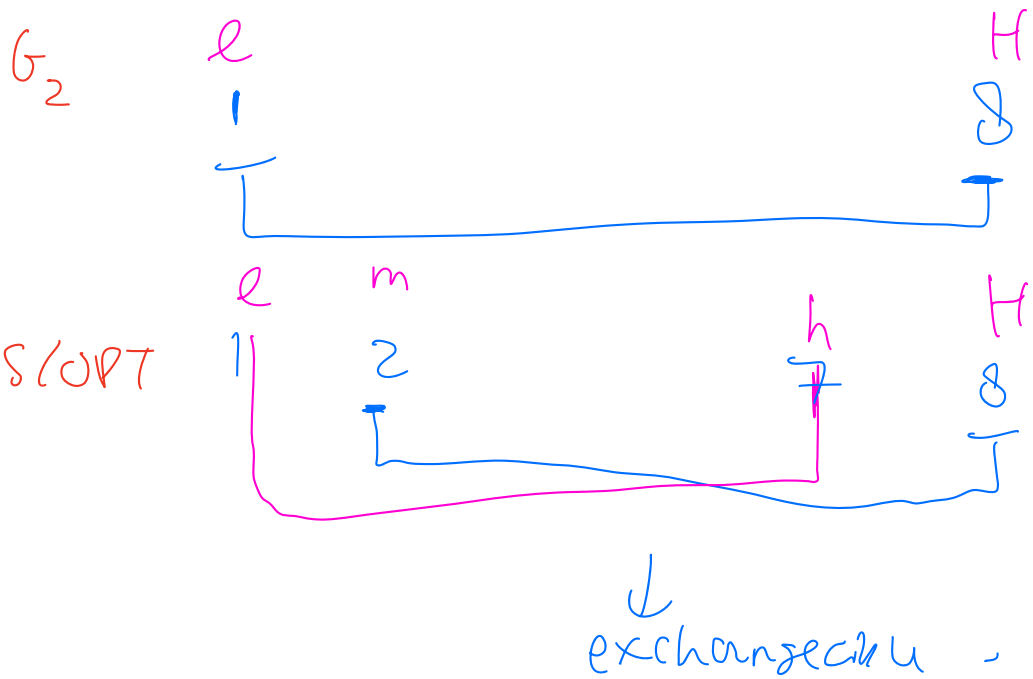
OPT' gives H greedy's partner instead



o ≤ g, so o + y ≤ g + y ≤ 10. And H + g fits, because greedy chose it.

because y ≤ H

Strat 2: H w/ e



↓

exchange ctrl.

tweak: H+e

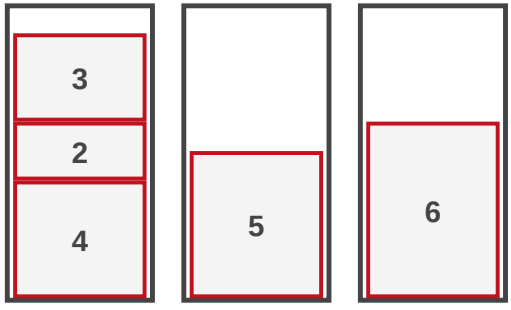
tweak: min ≤ mnt+1 ≤ limit

prev: optimal algo

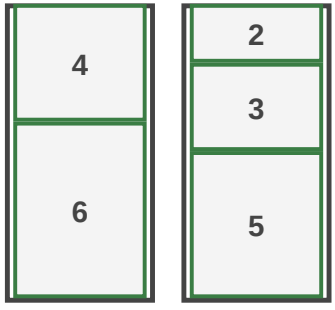
Approximation algorithms

Bin packing

Boxes [4, 2, 3, 5, 6], container capacity 10



3 containers



2 containers, and optimal

Formalized

Input: `weights`, an array of item sizes, and `capacity`, the size of one container.

Output: the minimum number of containers needed to hold every item.

Strategy

- open a new box
- in desc weight, put it in box until can't fit. item doesn't fit, so to next one

Strategy: first-fit decreasing

1. Sort the items **largest first**
2. For each item, put it in the first container it still fits in
3. If it fits nowhere, open a new container

Q: who runs? who's better?

Measuring quality of approx algos

Sorted largest first, **first**-fit decreasing:

optimal # boxes

$$\mathrm{FFD}(I) \leq \frac{11}{9}\mathrm{OPT}(I) + \frac{6}{9}$$

Unsorted, packing as they arrive:

$$\mathrm{FF}(I) \leq 1.7 \times \mathrm{OPT}(I)$$

More heuristics

Applying algos IRL

First attempt

- Full hard problem, not optimal,
- Clarify problem statement → research plans
- Implement a baseline algo — good enough & simple
  - → e.g. PTAS
- Maybe brute force if small.

01. ... cost, benefit

02. — — cost, benefit

03. Implement paper X → SoTA

03. use software package
use google OR-Tools

plan

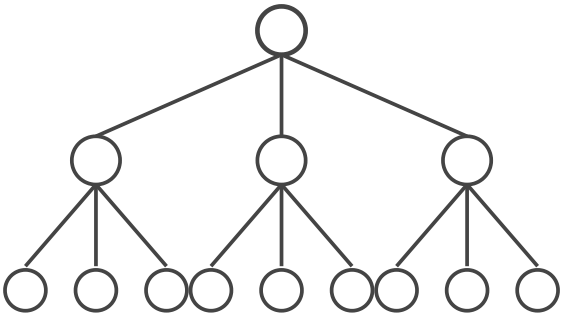
- pick one / two
- "implement"
- compare against baseline

just shop online. don't code it up from scratch

Recap: DP vs greedy

DP

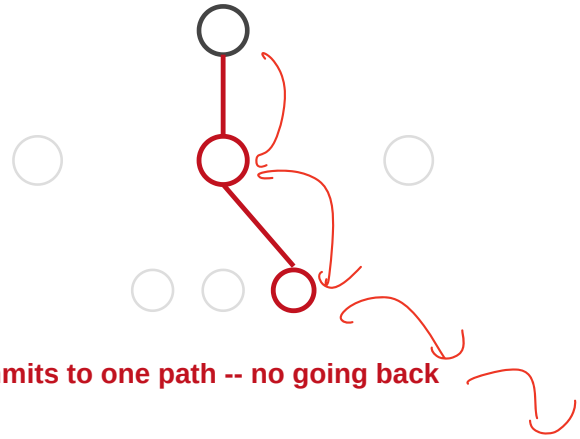
try every option, remember the answers



explores everything -- but caches

Greedy

one rule picks the move, then never look back

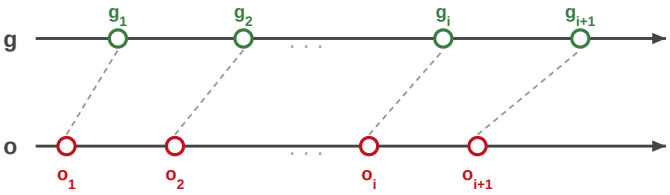


commits to one path -- no going back

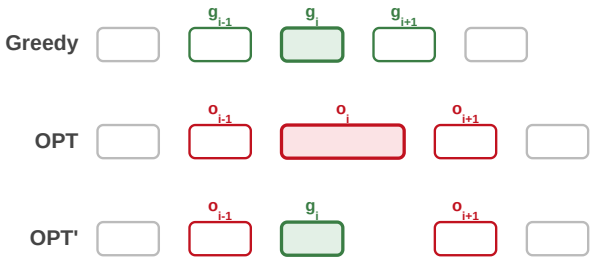
Recap: the cookbook

1. **Brainstorm** greedy strategies
2. **Prove** it, or kill it with a counterexample
3. **Code** it -- sort, then one sweep

Recap: Proof styles



Greedy stays ahead



Exchange